K-d trees are great, they offer a major speed-up over linear search; yet, they still have potential issues:

- K-d trees might be hard to implement efficiently;
- K-d trees are not self-balancing, so they perform best when they are constructed from a stable set of points, and the number of insert and remove is limited with respect to the total number of elements. Unfortunately, static datasets are not the norm in the big-data era.
- When we deal with high-dimensional spaces, k-d trees become inefficient: as we have seen, the time needed for removal and nearest neighbor search is exponential in k, the dimension of the dataset: but for sufficiently large values of k this hinder any performance benefit over brute-force search; we have seen that nearest neighbor search perform better than naïve search if $n > 2k$, so starting at $k \approx 30$ we would need an ideal dataset (one with a regular distribution of points) with billions of elements, in order for k-d tree to overperform brute-force.
- K-d trees doesn't work well with paged memory, they are not memory locally-efficient, as points are stored in tree nodes, so close-by points won't lie close-by memory areas.
- While they handle points well, k-d trees can't handle non-punctiform objects, like shapes or any object with non-zero measure.

The latter point stems from the fact that in high dimensional datasets data becomes very sparse; at the same time, when we traverse a k-d tree during NN-search, we can only prune a branch when it doesn't intersect the hyper-sphere centered